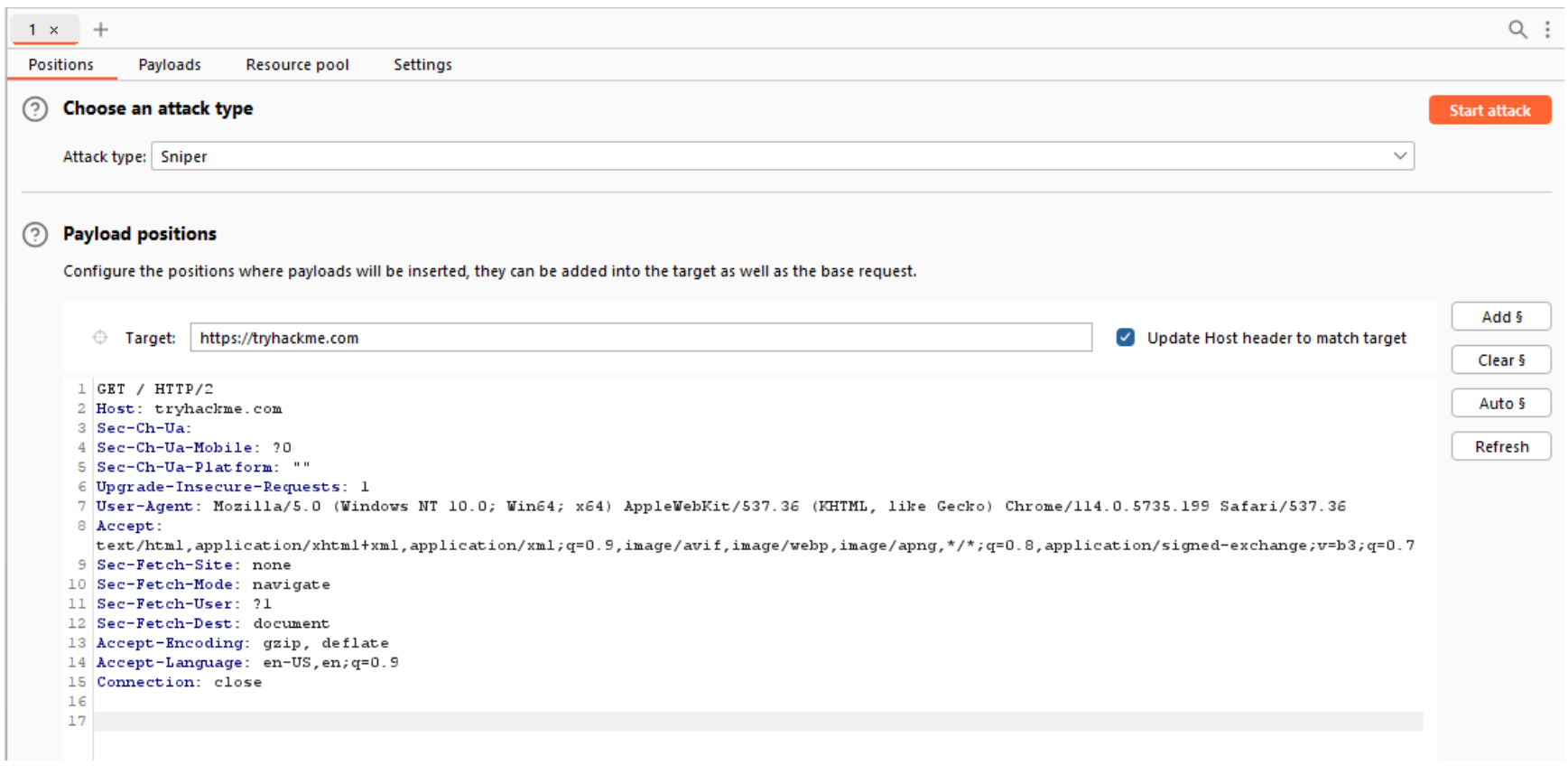
BURPSUITE INTRUDER

Burp Suite's Intruder module is a powerful tool that allows for automated and customisable attacks. It provides the ability to modify specific parts of a request and perform repetitive tests with variations of input data. Intruder is particularly useful for tasks like fuzzing and brute-forcing, where different values need to be tested against a target.

Intruder is Burp Suite's built-in fuzzing tool that allows for automated request modification and repetitive testing with variations in input values. By using a captured request (often from the Proxy module), Intruder can send multiple requests with slightly altered values based on user-defined configurations. It serves various purposes, such as brute-forcing login forms by substituting username and password fields with values from a wordlist or performing fuzzing attacks using wordlists to test subdirectories, endpoints, or virtual hosts. Intruder's functionality is comparable to command-line tools like **Wfuzz** or **ffuf**.

However, it's important to note that while Intruder can be used with Burp Community Edition, it is rate-limited, significantly reducing its speed compared to Burp Professional. This limitation often leads security practitioners to rely on other tools for fuzzing and brute-forcing. Nonetheless, Intruder remains a valuable tool and is worth learning how to use it effectively.

Let's explore the Intruder interface:



The initial view of Intruder presents a simple interface where we can select our target. This field will already be populated if a request has been sent from the Proxy (using `Ctrl + I` or right-clicking and selecting "Send to Intruder").

There are four sub-tabs within Intruder:

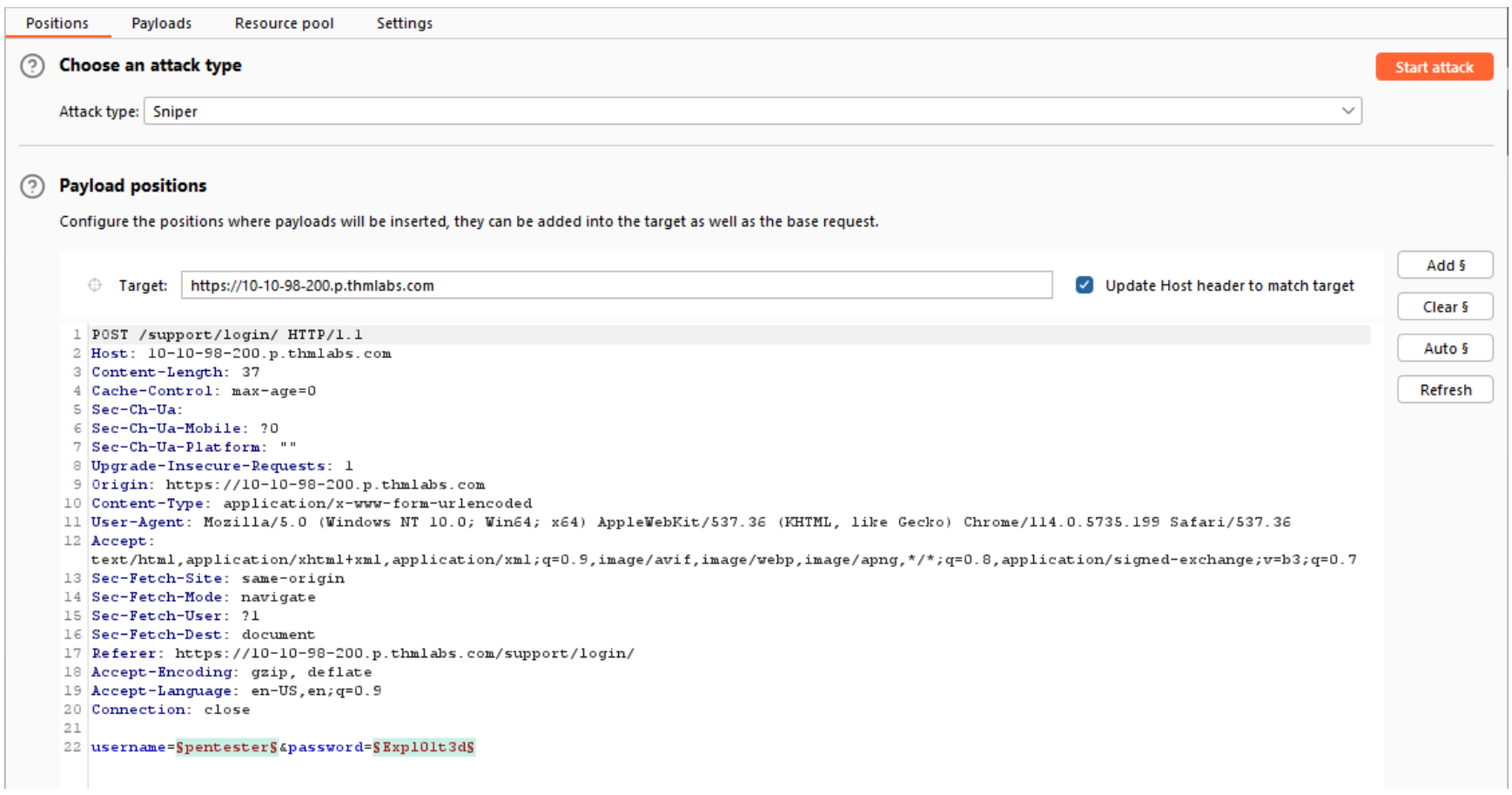
- **Positions**: This tab allows us to select an attack type (which we will cover in a future task) and configure where we want to insert our payloads in the request template.
- **Payloads**: Here we can select values to insert into the positions defined in the **Positions** tab. We have various payload options, such as loading items from a wordlist. The way these payloads are inserted into the template depends on the attack type chosen in the **Positions** tab. The **Payloads** tab also enables us to modify Intruder's behavior regarding payloads, such as defining pre-processing rules for each payload (e.g., adding a prefix or suffix, performing match and replace, or skipping payloads based on a defined regex).
- **Resource Pool**: This tab is not particularly useful in the Burp Community Edition. It allows for resource allocation among various automated tasks in Burp Professional. Without access to these automated tasks, this tab is of limited importance.
- **Settings**: This tab allows us to configure attack behavior. It primarily deals with how Burp handles results and the attack itself. For instance, we can flag requests containing specific text or define Burp's response to redirect (3xx) responses.

Note: The term "fuzzing" refers to the process of testing functionality or existence by applying a set of data to a parameter. For example, fuzzing for endpoints in a web application involves taking each word in a wordlist and appending it to a request URL (e.g., http://10.65.167.5/WORD_GOES_HERE) to observe the server's response.

When using Burp Suite Intruder to perform an attack, the first step is to examine the positions within the request where we want to insert our payloads.

When using Burp Suite Intruder to perform an attack, the first step is to examine the positions within the request where we want to insert our payloads. These positions inform Intruder about the locations where our payloads will be introduced (as we will explore in upcoming tasks).

Let's navigate to the Positions tab:

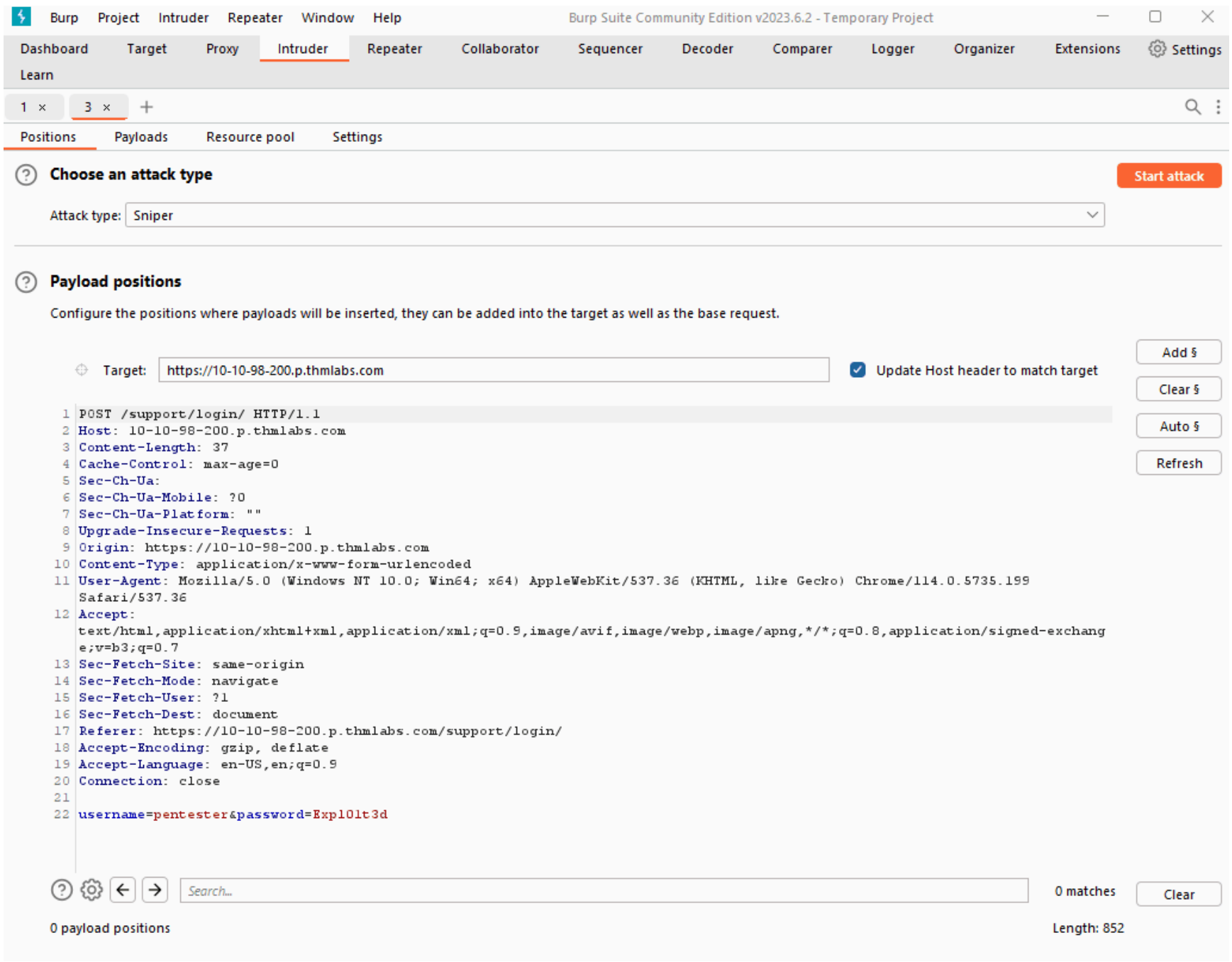


Notice that Burp Suite automatically attempts to identify the most probable positions where payloads can be inserted. These positions are highlighted in green and enclosed by section marks (`§`).

On the right-hand side of the interface, we find the following buttons: `Add §`, `Clear §`, and `Auto §`:

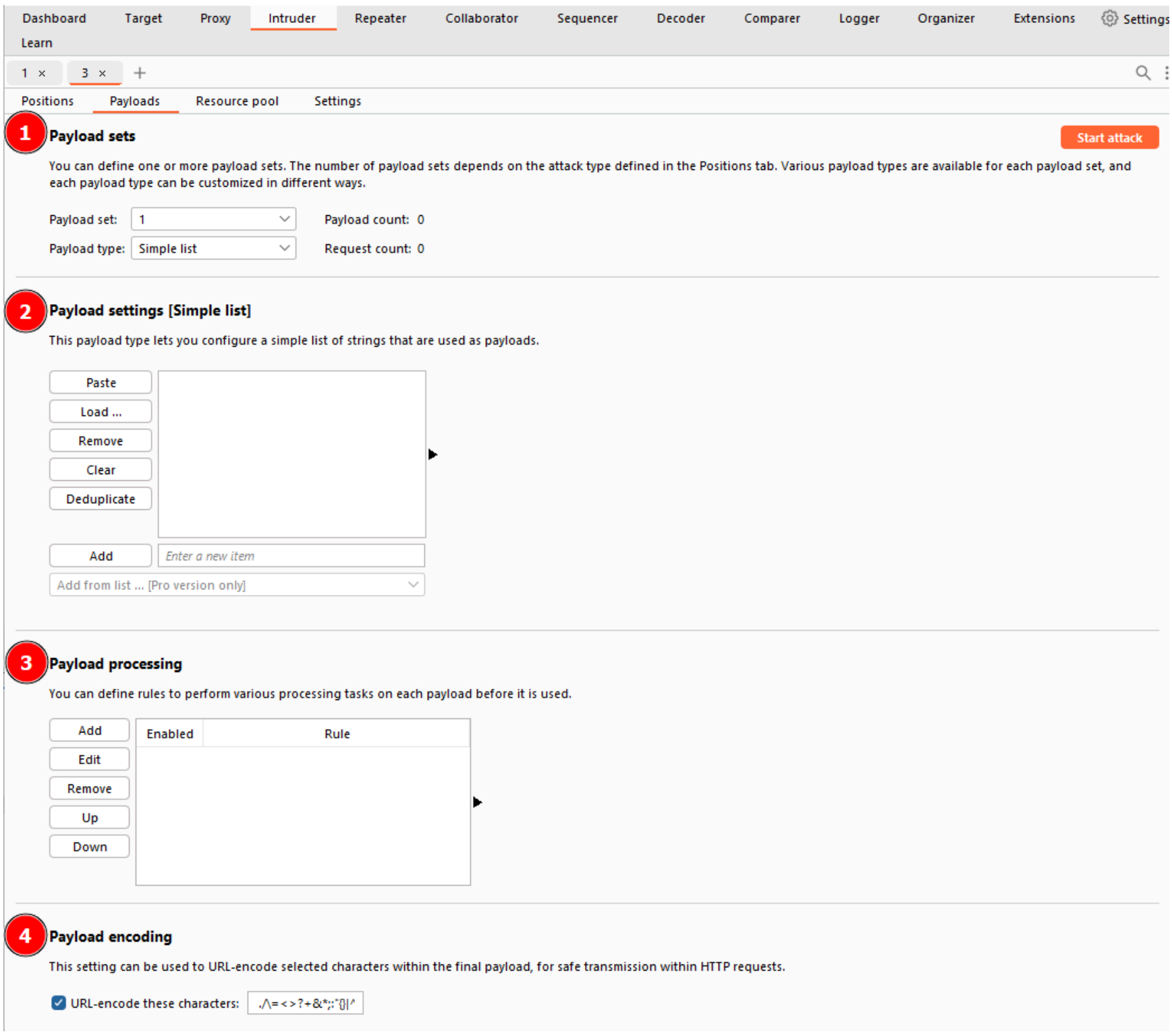
- The `Add §` button allows us to define new positions manually by highlighting them within the request editor and then clicking the button.
- The `Clear §` button removes all defined positions, providing a blank canvas where we can define our own positions.
- The `Auto §` button automatically attempts to identify the most likely positions based on the request. This feature is helpful if we previously cleared the default positions and want them back.

The following GIF demonstrates the process of adding, clearing, and automatically reselecting positions:

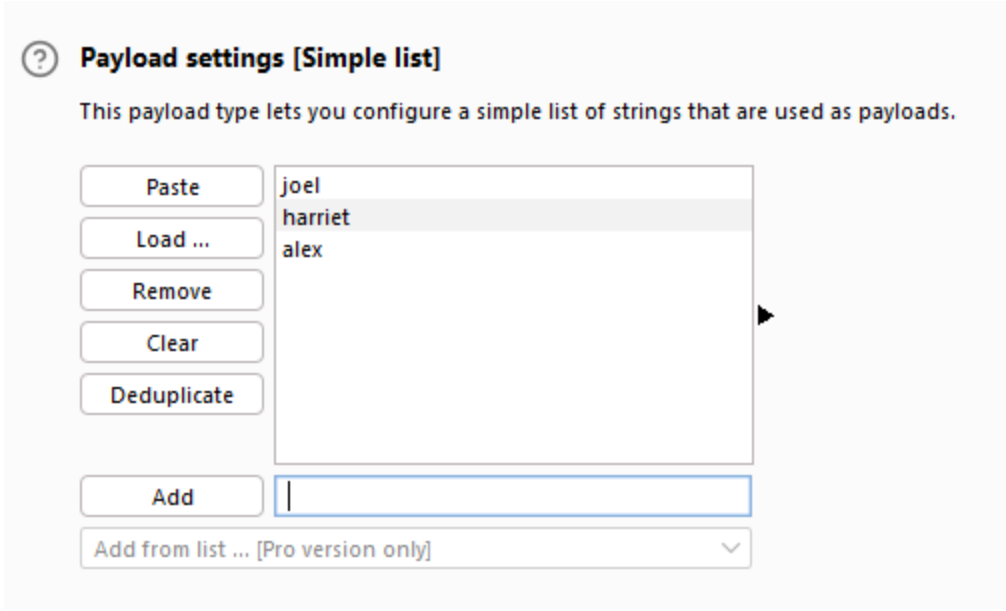


Take some time to familiarize yourself with adding, clearing, and auto-selecting positions using the Burp Suite Intruder interface.

In the **Payloads** tab of Burp Suite Intruder, we can create, assign, and configure payloads for our attack. This sub-tab is divided into four sections:



1. **Payload Sets**:
   - This section allows us to choose the position for which we want to configure a payload set and select the type of payload we want to use.
   - When using attack types that allow only a single payload set (Sniper or Battering Ram), the "Payload Set" dropdown will have only one option, regardless of the number of defined positions.
   - If we use attack types that require multiple payload sets (Pitchfork or Cluster Bomb), there will be one item in the dropdown for each position.
   - **Note:** When assigning numbers in the "Payload Set" dropdown for multiple positions, follow a top-to-bottom, left-to-right order. For example, with two positions (`username=§pentester§&password=§Expl01ted§`), the first item in the payload set dropdown would refer to the username field, and the second item would refer to the password field.
2. **Payload settings**:
   - This section provides options specific to the selected payload type for the current payload set.
   - For example, when using the "Simple list" payload type, we can manually add or remove payloads to/from the set using the **Add** text box, **Paste** lines, or **Load** payloads from a file. The **Remove** button removes the currently selected line, and the **Clear** button clears the entire list. Be cautious with loading huge lists, as it may cause Burp to crash.
   - Each payload type will have its own set of options and functionality. Explore the options available to understand the range of possibilities.



3. **Payload Processing**:
   - In this section, we can define rules to be applied to each payload in the set before it is sent to the target.
   - For example, we can capitalize every word, skip payloads that match a regex pattern, or apply other transformations or filtering.
   - While you may not use this section frequently, it can be highly valuable when specific payload processing is required for your attack.
4. **Payload Encoding**:
   - The section allows us to customize the encoding options for our payloads.
   - By default, Burp Suite applies URL encoding to ensure the safe transmission of payloads. However, there may be cases where we want to adjust the encoding behavior.
   - We can override the default URL encoding options by modifying the list of characters to be encoded or unchecking the "URL-encode these characters" checkbox.

By leveraging these sections, we can create and customise payload sets to suit the specific requirements of our attacks. This level of control allows us to finely tune our payloads for effective testing and exploitation.

The **Positions** tab of Burp Suite Intruder has a dropdown menu for selecting the attack type. Intruder offers four attack types, each serving a specific purpose. Let's explore each of them:

1. **Sniper**: The Sniper attack type is the default and most commonly used option. It cycles through the payloads, inserting one payload at a time into each position defined in the request. Sniper attacks iterate through all the payloads in a linear fashion, allowing for precise and focused testing.
2. **Battering ram**: The Battering ram attack type differs from Sniper in that it sends all payloads simultaneously, each payload inserted into its respective position. This attack type is useful when testing for race conditions or when payloads need to be sent concurrently.
3. **Pitchfork**: The Pitchfork attack type enables the simultaneous testing of multiple positions with different payloads. It allows the tester to define multiple payload sets, each associated with a specific position in the request. Pitchfork attacks are effective when there are distinct parameters that need separate testing.
4. **Cluster bomb**: The Cluster bomb attack type combines the Sniper and Pitchfork approaches. It performs a Sniper-like attack on each position but simultaneously tests all payloads from each set. This attack type is useful when multiple positions have different payloads, and we want to test them all together.

Each attack type has its advantages and is suitable for different testing scenarios. Understanding their differences helps us select the appropriate attack type based on the testing objectives.

The **Sniper** attack type is the default and most commonly used attack type in Burp Suite Intruder. It is particularly effective for single-position attacks, such as password brute-force or fuzzing for API endpoints. In a Sniper attack, we provide a set of payloads, which can be a wordlist or a range of numbers, and Intruder inserts each payload into each defined position in the request.

Let's refer to our example template from before:

Example Positions

```
POST /support/login/ HTTP/1.1
Host: 10.65.167.5
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:80.0) Gecko/20100101 Firefox/8
0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 37
Origin: http://10.65.167.5
Connection: close
Referer: http://10.65.167.5/support/login/
Upgrade-Insecure-Requests: 1

username=§pentester§&password=§Expl01ted§
```

In this example, we have two positions defined for the `username` and `password` body parameters. In a Sniper attack, Intruder takes each payload from the payload set and substitutes it into each defined position in turn.

Assuming we have a wordlist with three words: `burp`, `suite`, and `intruder`, Intruder would generate six requests:

Request Number	Request Body
1	`username=burp&password=Expl01ted`
2	`username=suite&password=Expl01ted`
3	`username=intruder&password=Expl01ted`
4	`username=pentester&password=burp`
5	`username=pentester&password=suite`
6	`username=pentester&password=intruder`

Observe how Intruder starts with the first position (`username`) and substitutes each payload into it, then moves to the second position (`password`) and performs the same substitution with the payloads. The total number of requests made by Intruder Sniper can be calculated as `requests = numberOfWords * numberOfPositions`.

The Sniper attack type is beneficial when we want to perform tests with single-position attacks, utilizing different payloads for each position. It allows for precise testing and analysis of different payload variations.

The **Battering ram** attack type in Burp Suite Intruder differs from Sniper in that it places the same payload in every position simultaneously, rather than substituting each payload into each position in turn.

Let's refer back to our previous example template:

Example Positions

```
POST /support/login/ HTTP/1.1
    Host: 10.65.167.5
    User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:80.0) Gecko/20100101 Firefo
x/80.0
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
    Accept-Language: en-US,en;q=0.5
    Accept-Encoding: gzip, deflate
    Content-Type: application/x-www-form-urlencoded
    Content-Length: 37
    Origin: http://10.65.167.5
    Connection: close
    Referer: http://10.65.167.5/support/login/
    Upgrade-Insecure-Requests: 1

    username=§pentester§&password=§Expl01ted§
```

Using the Battering Ram attack type with the same wordlist from before (`burp`, `suite`, and `intruder`), Intruder would generate three requests:

Request Number	Request Body
1	`username=burp&password=burp`
2	`username=suite&password=suite`
3	`username=intruder&password=intruder`

As shown in the table, each payload from the wordlist is inserted into every position for each request made. In a Battering Ram attack, the same payload is thrown at every defined position simultaneously, providing a brute-force-like approach to testing.

The Battering Ram attack type is useful when we want to test the same payload against multiple positions at once without the need for sequential substitution.

The **Pitchfork** attack type in Burp Suite Intruder is similar to having multiple Sniper attacks running simultaneously. While Sniper uses one payload set to test all positions simultaneously, Pitchfork utilises one payload set per position (up to a maximum of 20) and iterates through them all simultaneously.

To better understand Pitchfork, let us revisit our brute-force example, but this time with two wordlists:

1. The first wordlist contains usernames: `joel`, `harriet`, and `alex`.
2. The second wordlist contains passwords: `J03l`, `Emma1815`, and `Sk1ll`.

We can use these two lists to perform a Pitchfork attack on the login form. Each request made during the attack would look like this:

Request Number	Request Body
1	`username=joel&password=J03l`
2	`username=harriet&password=Emma1815`
3	`username=alex&password=Sk1ll`

As shown in the table, Pitchfork takes the first item from each list and substitutes them into the request, one per position. It then repeats this process for the next request by taking the second item from each list and substituting it into the template. Intruder continues this iteration until one or all of the lists run out of items. It's important to note that Intruder stops testing as soon as one of the lists is complete. Therefore, in Pitchfork attacks, it is ideal for the payload sets to have the same length. If the lengths of the payload sets differ, Intruder will only make requests until the shorter list is exhausted, and the remaining items in the longer list will not be tested.

The Pitchfork attack type is especially useful when conducting credential-stuffing attacks or when multiple positions require separate payload sets. It allows for simultaneous testing of multiple positions with different payloads.

The **Cluster bomb** attack type in Burp Suite Intruder allows us to choose multiple payload sets, one per position (up to a maximum of 20). Unlike Pitchfork, where all payload sets are tested simultaneously, Cluster bomb iterates through each payload set individually, ensuring that every possible combination of payloads is tested.

To illustrate the Cluster bomb attack type, let's use the same wordlists as before:

- Usernames: `joel`, `harriet`, and `alex`.
- Passwords: `J03l`, `Emma1815`, and `Sk1ll`.

In this example, let's assume that we don't know which password belongs to which user. We have three users and three passwords, but the mappings are unknown. In this case, we can use a Cluster bomb attack to try every combination of values. The request table for our username and password positions would look like this:

Request Number	Request Body
1	`username=joel&password=J03l`
2	`username=harriet&password=J03l`
3	`username=alex&password=J03l`
4	`username=joel&password=Emma1815`
5	`username=harriet&password=Emma1815`

Request Number	Request Body
6	`username=alex&password=Emma1815`
7	`username=joel&password=Sk1ll`
8	`username=harriet&password=Sk1ll`
9	`username=alex&password=Sk1ll`

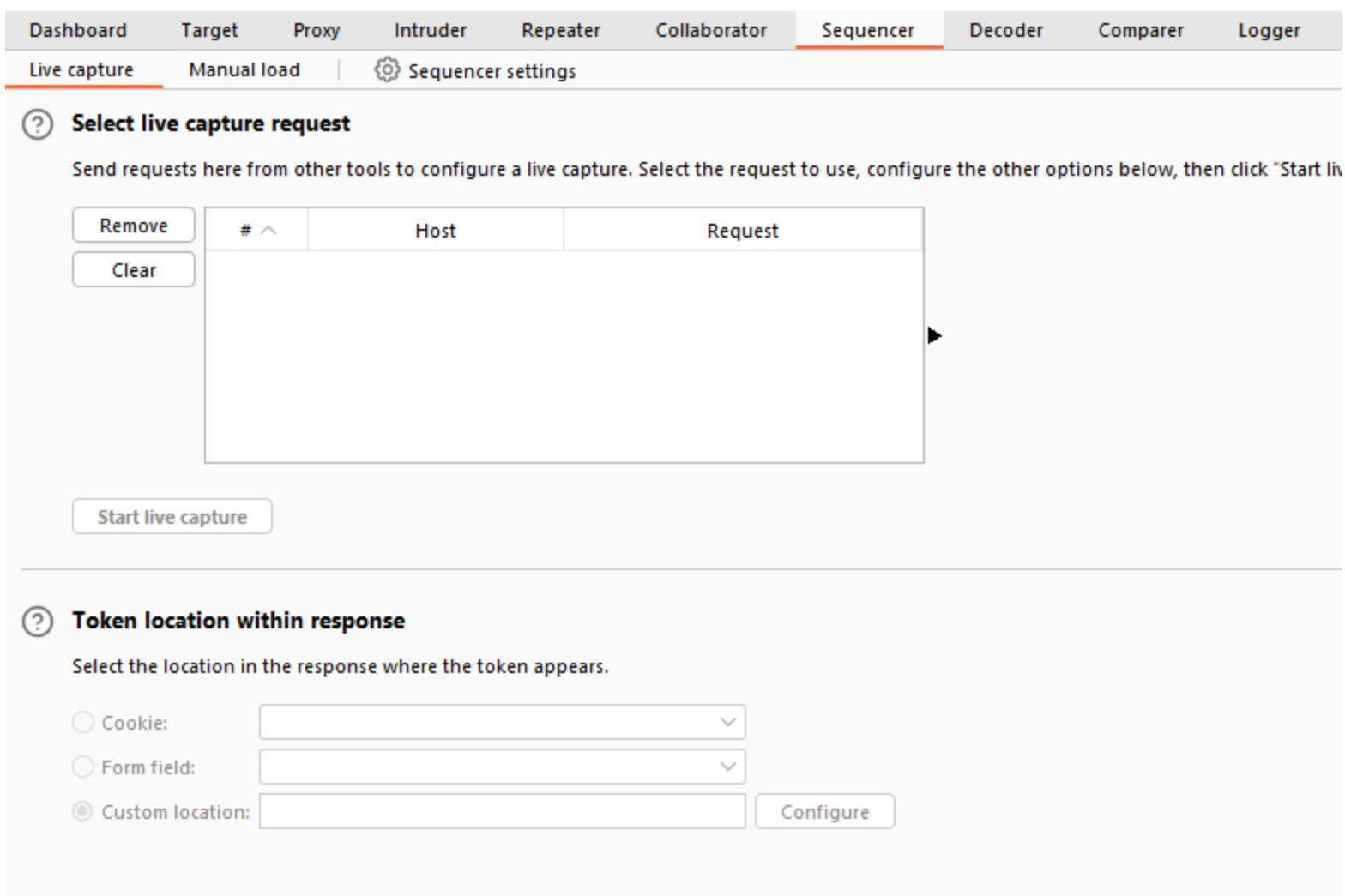
As shown in the table, the Cluster bomb attack type iterates through every combination of the provided payload sets. It tests every possibility by substituting each value from each payload set into the corresponding position in the request.

Cluster bomb attacks can generate a significant amount of traffic as it tests every combination. The number of requests made by a Cluster bomb attack can be calculated by multiplying the number of lines in each payload set together. It's important to be cautious when using this attack type, especially when dealing with large payload sets. Additionally, when using Burp Community and its Intruder rate-limiting, the execution of a Cluster bomb attack with a moderately sized payload set can take a significantly longer time.

The Cluster bomb attack type is particularly useful for credential brute-forcing scenarios where the mapping between usernames and passwords is unknown.

Sequencer allows us to evaluate the entropy, or randomness, of "tokens". Tokens are strings used to identify something and should ideally be generated in a cryptographically secure manner. These tokens could be session cookies or **C**ross-**S**ite **R**equest **F**orgery (CSRF) tokens used to protect form submissions. If these tokens aren't generated securely, then, in theory, we could predict upcoming token values. The implications could be substantial, for instance, if the token in question is used for password resets.

Let's start by looking at the Sequencer interface:

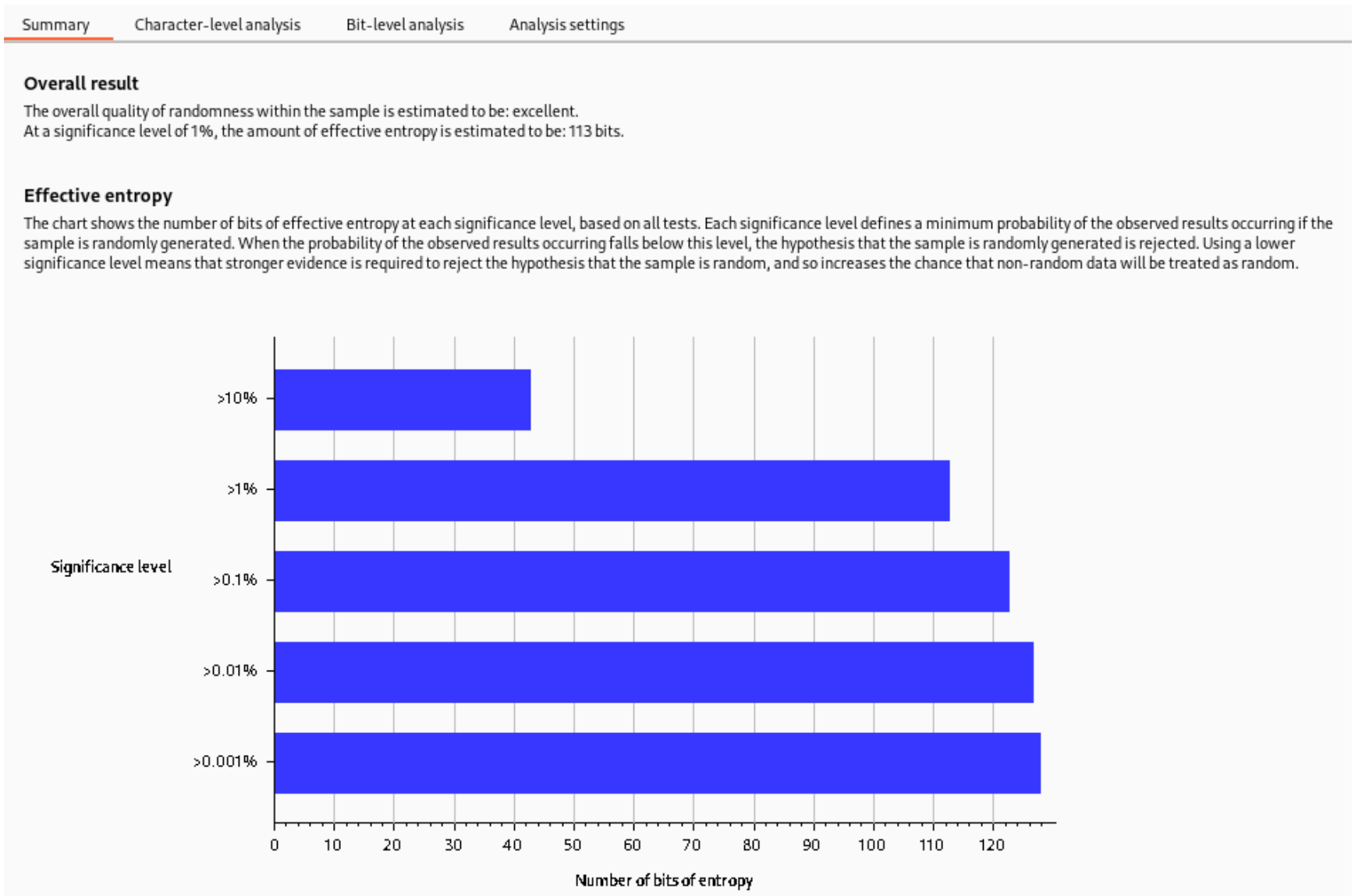


We have two main ways to perform token analysis with Sequencer:

- **Live Capture**: This is the more common method and is the default sub-tab for Sequencer. Live capture lets us pass a request that will generate a token to Sequencer for analysis. For instance, we might want to pass a POST request to a login endpoint to Sequencer, knowing that the server will respond with a cookie. With the request passed in, we can instruct Sequencer to start a live capture. It will then automatically make the same request thousands of times, storing the generated token samples for analysis. After collecting enough samples, we stop the Sequencer and allow it to analyze the captured tokens.

- **Manual Load**: This allows us to load a list of pre-generated token samples directly into Sequencer for analysis. Using Manual Load means we don't need to make thousands of requests to our target, which can be noisy and resource-intensive. However, it does require that we have a large list of pre-generated tokens.

The generated entropy analysis report is split into four primary sections. The first of these is the **Summary** of the results. The summary gives us the following:



- **Overall result**: This gives a broad assessment of the security of the token generation mechanism. In this case, the level of entropy indicates that the tokens are likely securely generated.
- **Effective entropy**: This measures the randomness of the tokens. The effective entropy of 117 bits is relatively high, indicating that the tokens are sufficiently random and, therefore, secure against prediction or brute force attacks.
- **Reliability**: The significance level of 1% implies that there is 99% confidence in the accuracy of the results. This level of confidence is quite high, providing assurance in the accuracy of the effective entropy estimation.
- **Sample**: This provides details about the token samples analyzed during the entropy testing process, including the number of tokens and their characteristics.

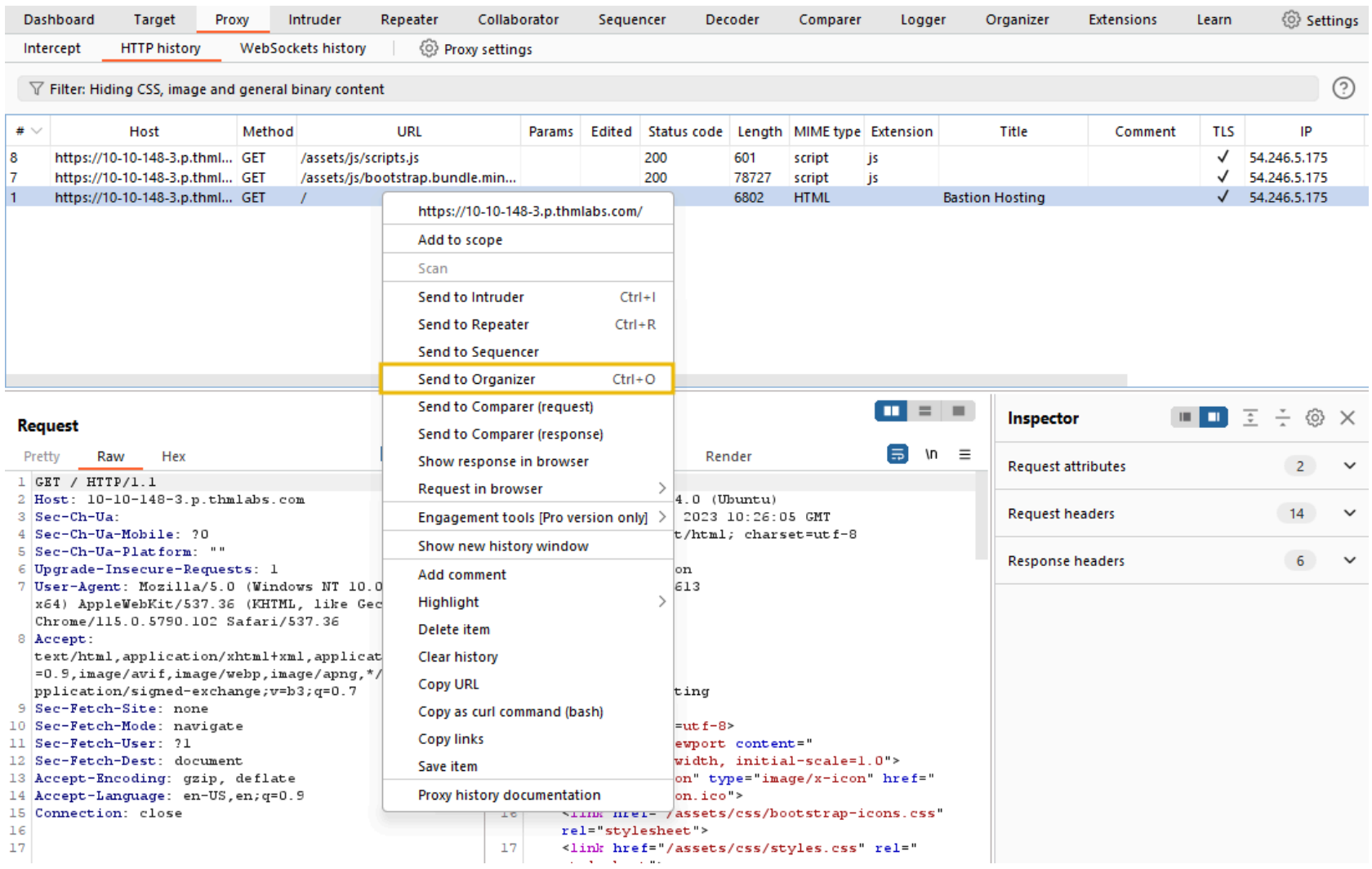
While the summary report often provides enough information to assess the security of the token generation process, it's important to remember that further investigation may be necessary in some cases. The character-level and bit-level analysis can provide more detailed insights into the randomness of the tokens, especially when the summary results raise potential concerns.

While the entropy report can provide a strong indicator of the security of the token generation mechanism, there needs to be more definitive proof. Other factors could also impact the security of the tokens, and the nature of probability and statistics means there's always a degree of uncertainty. That said, an effective entropy of 117 bits with a significance level of 1% suggests a robustly secure token generation process.

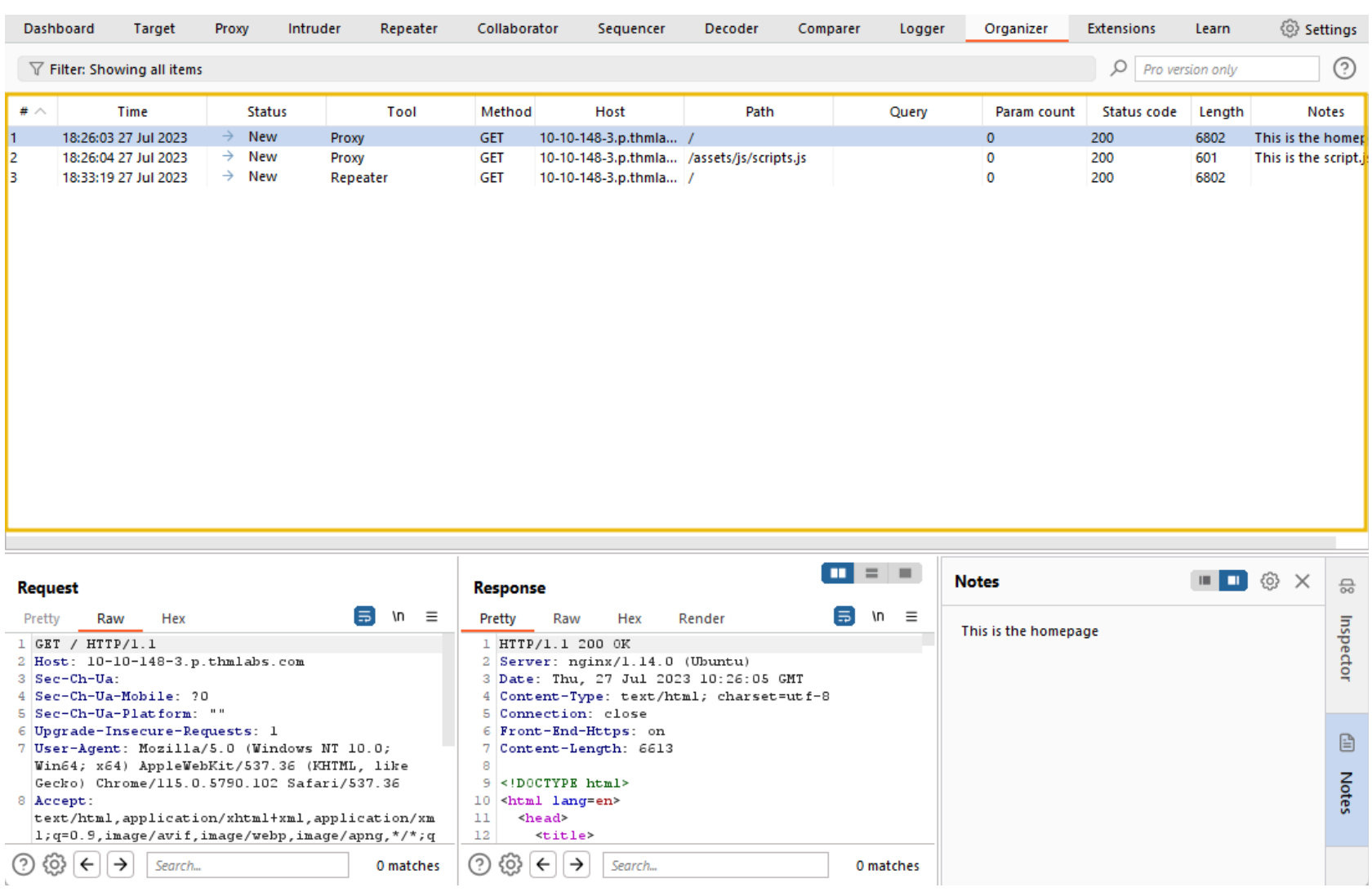
The Organizer module of Burp Suite is designed to help you store and annotate copies of HTTP requests that you may want to revisit later. This tool can be particularly useful for organizing your penetration testing workflow. Here are some of its key features:

- You can store requests that you want to investigate later, save requests that you've already identified as interesting, or save requests that you want to add to a report later.

- You can send HTTP requests to Burp Organizer from other Burp Modules such as **Proxy** or **Repeater**. You can do this by right-clicking the request and selecting **Send to Organizer** or using the default hotkey `Ctrl + O`. Each HTTP request that you send to Organizer is a read-only copy of the original request saved at the point you sent it to Organizer.



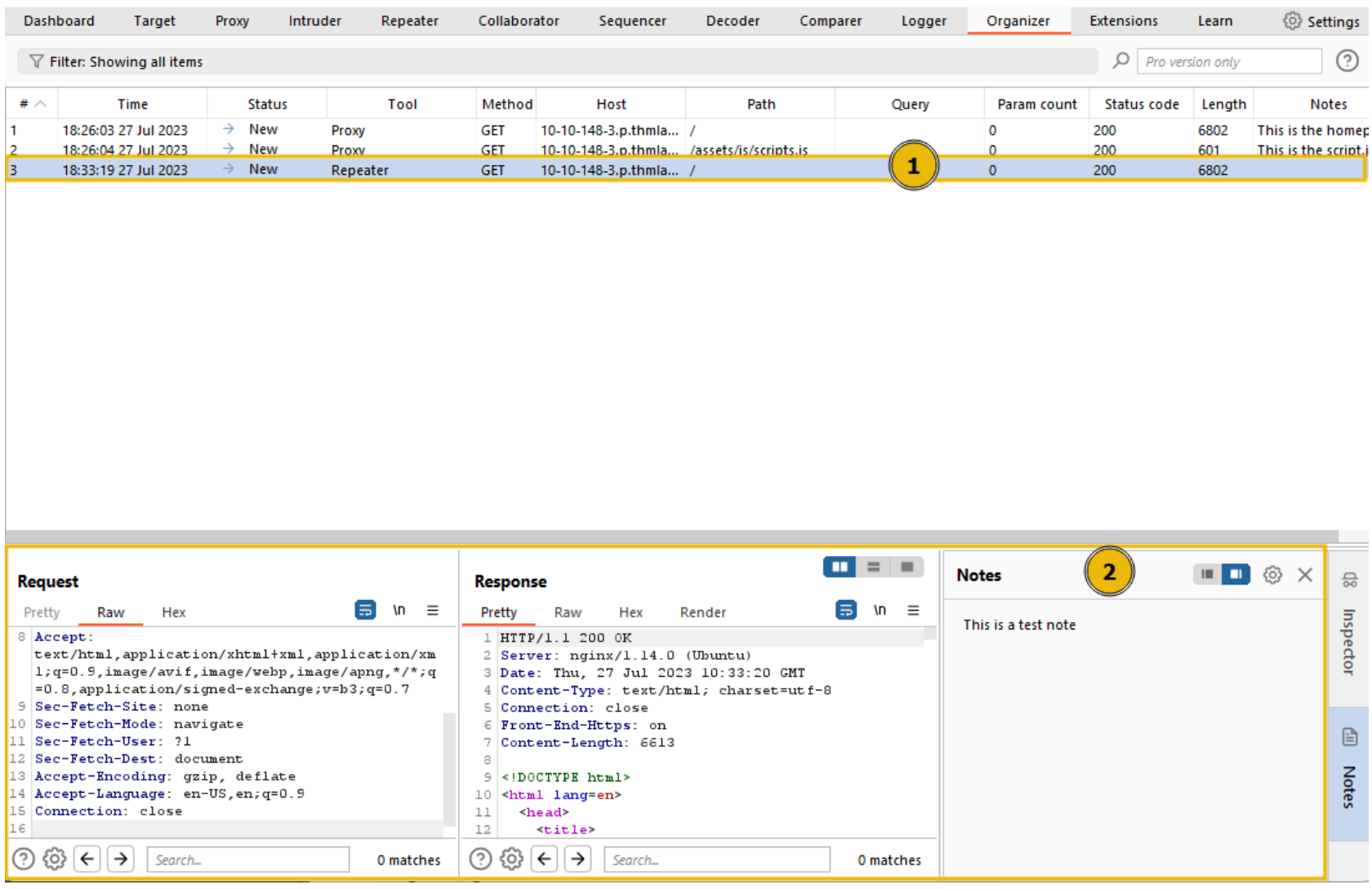
- Requests are stored in a table, which contains columns such as the request index number, the time the request was made, workflow status, Burp tool that the request was sent from, HTTP method, server hostname, URL file path, URL query string, number of parameters in the request, HTTP status code of the response, length of the response in bytes, and any notes that you have made.



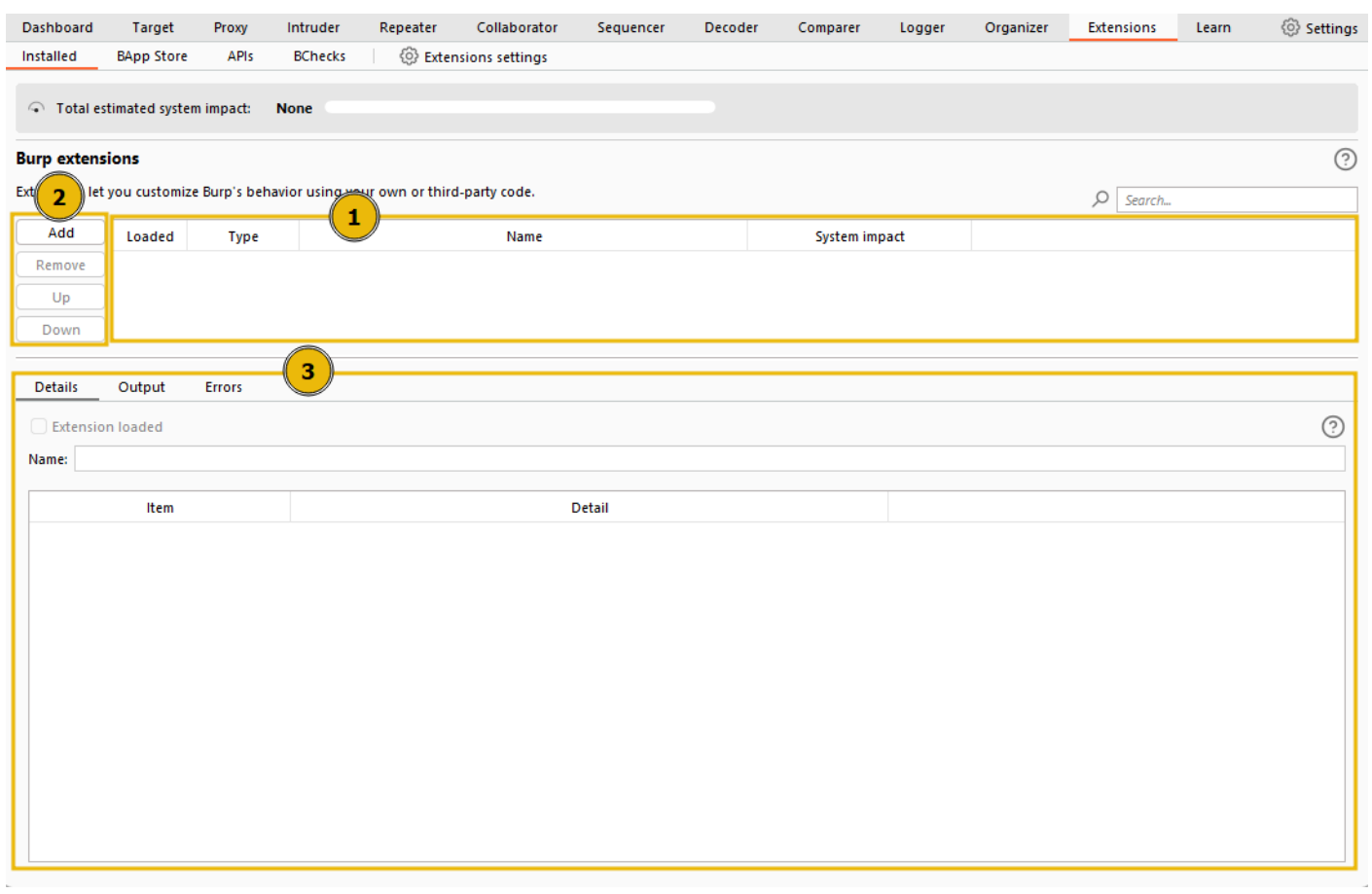
To view the request and response:

1. Click on any Organizer item.

2. The request and response are both read-only. You can search within the request or response, select the request, and then use the search bar below the request.



The Extensions interface in Burp Suite provides an overview of the extensions loaded into the tool. Let's take a look at the different components of this interface:



1. **Extensions List**: The top box displays a list of the extensions that are currently installed in Burp Suite for the current project. It allows you to activate or deactivate individual extensions.
2. **Managing Extensions**: On the left side of the Extensions interface, there are options to manage extensions:
   - **Add**: You can use this button to install new extensions from files on your disk. These files can be custom-coded modules or modules obtained from external sources that are not available in the official BApp store.
   - **Remove**: This button allows you to uninstall selected extensions from Burp Suite.

- **Up/Down**: These buttons control the order in which installed extensions are listed. The order determines the sequence in which extensions are invoked when processing traffic. Extensions are applied in descending order, starting from the top of the list and moving down. The order is essential, especially when dealing with extensions that modify requests, as some may conflict or interfere with others.

3. **Details, Output, and Errors**: Towards the bottom of the window, there are sections for the currently selected extension:
   - **Details**: This section provides information about the selected extension, such as its name, version, and description.
   - **Output**: Extensions can produce output during their execution, and this section displays any relevant output or results.
   - **Errors**: If an extension encounters any errors during execution, they will be shown in this section. This can be useful for debugging and troubleshooting extension issues.

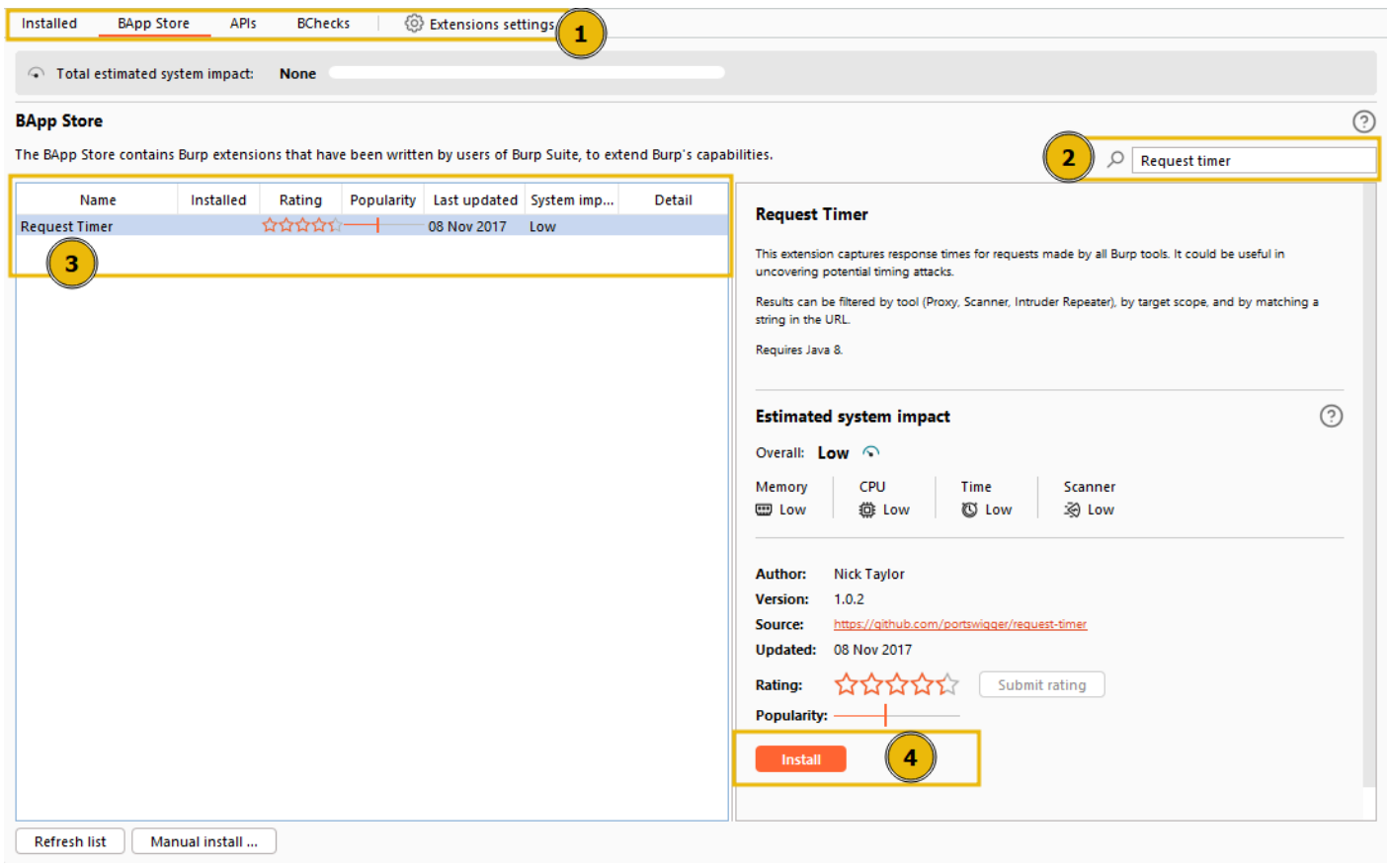
In summary, the Extensions interface in Burp Suite allows users to manage and monitor the installed extensions, activate or deactivate them for specific projects, and view important details, output, and errors related to each extension. By using extensions, Burp Suite becomes a powerful and customizable platform for various security testing and web application assessment tasks.

In Burp Suite, the BApp Store (Burp App Store) allows us to easily discover and integrate official extensions seamlessly into the tool. Extensions can be written in various languages, with Java and Python being the most common choices. Java extensions integrate automatically with the Burp Suite framework, while Python extensions require the Jython interpreter.

To get a feel for the BApp store and install a Java extension, let's install the Request Timer extension, written by Nick Taylor. The Request Timer extension enables us to log the time it takes for each request to receive a response. This functionality is particularly useful for identifying and exploiting time-based vulnerabilities. For instance, if a login form takes an extra second to process requests with valid usernames compared to invalid ones, we can use the time differences to determine which usernames are valid.

Follow these steps to install the Request Timer extension from the BApp store:

1. Switch to the **BApp Store** sub-tab in Burp Suite.
2. Use the search function to find **Request Timer**. There should only be one result for this extension.
3. Click on the returned extension to view more details.
4. Click the **Install** button to install the Request Timer extension.



After successfully installing the extension, you will notice a new tab appearing in the main menu at the top of the Burp Suite interface. Different extensions may have varying behaviours. Some may add new items to right-click context menus, while others create entirely new tabs in the main menu bar.

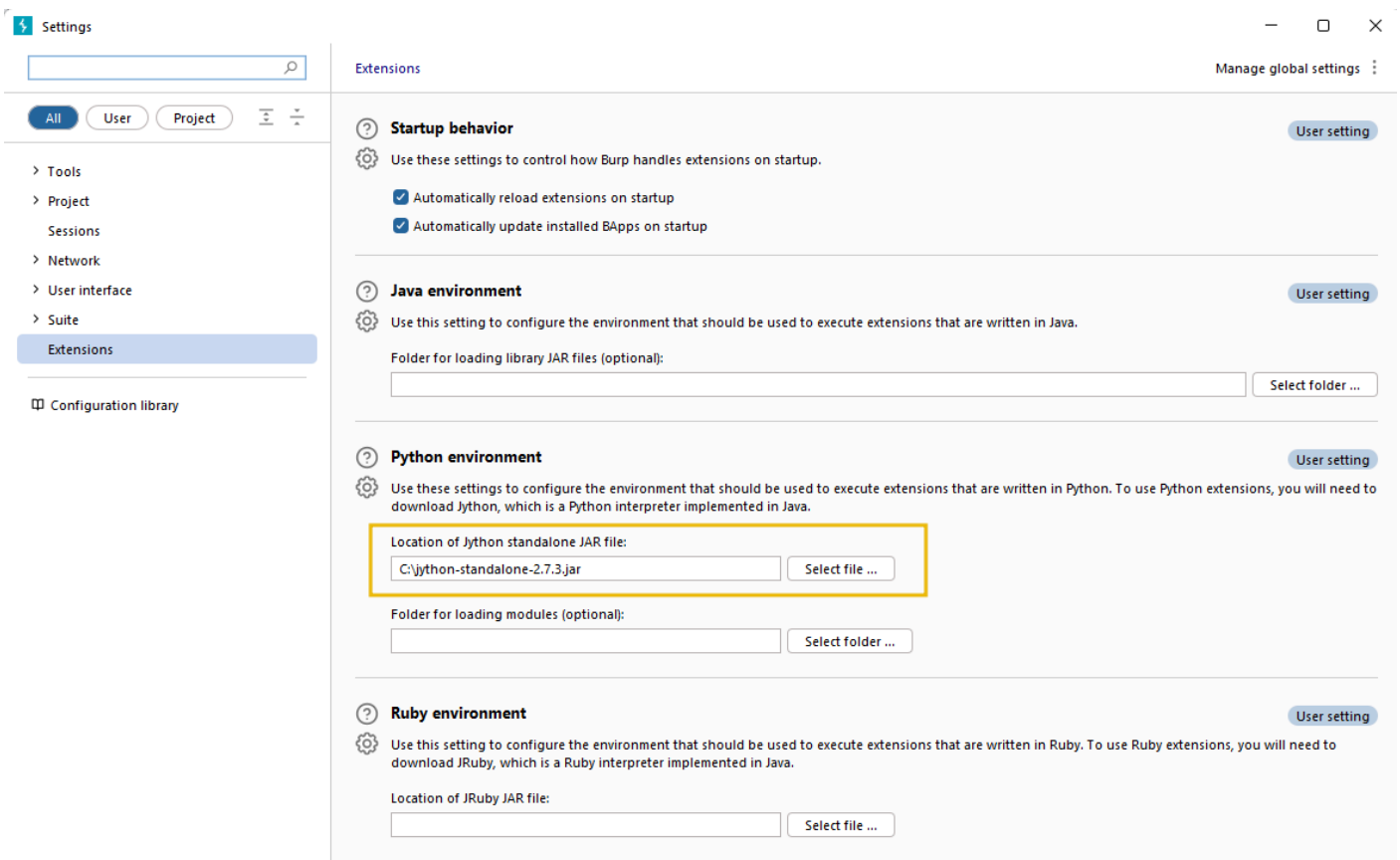
As this installation is simply an example of using the BApp store, we won't cover how to use the Request Timer here. However, I highly recommend switching to the new tab and exploring the extension to understand its functionalities better. The Request Timer can be valuable in various scenarios, especially when assessing web application security and identifying potential time-based vulnerabilities.

To use Python modules in Burp Suite, we need to include the Jython Interpreter JAR file, which is a Java implementation of Python. The Jython Interpreter enables us to run Python-based extensions within Burp Suite.

Follow these steps to integrate Jython into Burp Suite on your local machine:

1. **Download Jython JAR**: Visit the Jython website and download the standalone JAR archive. Look for the **Jython Standalone** option. Save the JAR file to a location on your disk.

2. **Configure Jython in Burp Suite**: Open Burp Suite and switch to the **Extensions** module. Then, go to the **Extensions settings** sub-tab.
3. **Python Environment**: Scroll down to the "Python environment" section.
4. **Set Jython JAR Location**: In the "Location of Jython standalone JAR file" field, set the path to the downloaded Jython JAR file.

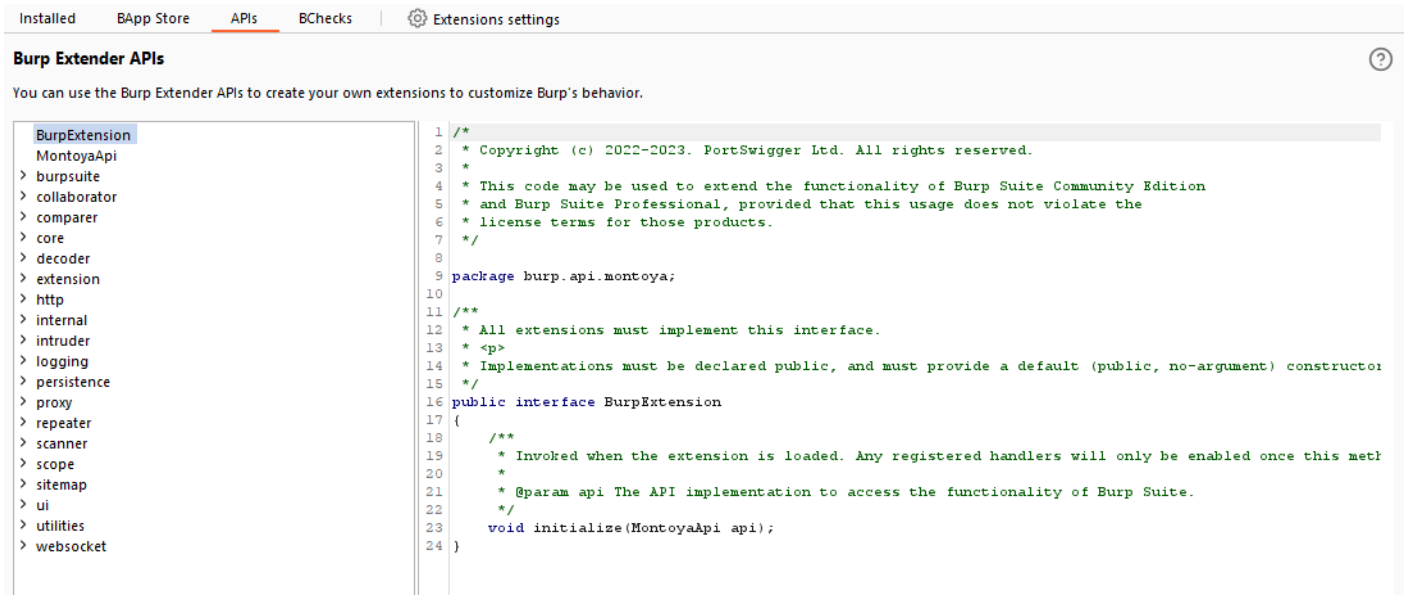


Once you have completed these steps, Jython will be integrated with Burp Suite, allowing you to use Python modules in the tool. This integration significantly increases the number of available extensions and enhances your capabilities in performing various security testing and web application assessment tasks.

Note: The process of adding Jython to Burp Suite is the same for all operating systems, as Java is a multi-platform technology.

In the Burp Suite Extensions module, you have access to a wide range of API endpoints that allow you to create and integrate your modules with Burp Suite. These APIs expose various functionalities, enabling you to extend the capabilities of Burp Suite to suit your specific needs.

To view the available API endpoints, navigate to the **APIs** sub-tab within the Extensions module. Each item listed in the left-hand panel represents a different API endpoint that can be accessed from within extensions.



The Extensions APIs give developers significant power and flexibility when writing custom extensions. You can use these APIs to seamlessly interact with Burp Suite's existing functionality and tailor your extensions to perform specific tasks.

Burp Suite supports multiple languages for writing extensions, such as:

1. Java (Natively): You can directly use Java to write extensions for Burp Suite, taking advantage of the powerful APIs available.
2. Python (via Jython): If you prefer Python as your programming language, you can utilize Jython, which is a Java implementation of Python to create Burp Suite extensions.
3. Ruby (via JRuby): Developers familiar with Ruby can leverage JRuby, a Java implementation of Ruby, to build Burp Suite extensions.

It's important to note that coding your extensions for Burp Suite can be a complex task, and it goes beyond the scope of this module. However, suppose you are interested in exploring this area further and creating custom extensions. In that case, PortSwigger provides a comprehensive reference that is an excellent resource for developing Burp Suite extensions.

To learn more about Burp Suite extension development and to access the detailed reference, you can visit PortSwigger's official documentation. This resource will provide the information and guidance needed to create powerful and customized extensions that enhance your experience with Burp Suite.